

```
package jp.co.activekenshu.delivery.servlet.customer;
```

```
import java.io.IOException;
```

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import jp.co.activekenshu.delivery.dto.OrderDto;
```

```
/**
```

```
 * 配送依頼情報をDBに登録するServletです。
```

```
 *
```

```
 * <p>
```

```
 * 本来はServiceやRepositoryに分ける処理も、
```

```
 * 今回はServlet内にまとめて書いています。
```

```
 * </p>
```

```
 */
```

```
@WebServlet("/customer/order/register")
```

```
public class RegisterServlet extends HttpServlet {
```

```
    /** DB接続URLです。環境に合わせて変更してください。 */
```

```
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:XE";
```

```
    /** DBユーザー名です。 */
```

```
    private static final String USER = "ユーザー名";
```

```
    /** DBパスワードです。 */
```

```
    private static final String PASSWORD = "パスワード";
```

```
    /** 名前項目の最大文字数です。 */
```

```
    private static final int NAME_MAX_LENGTH = 120;
```

```
    /** 住所項目の最大文字数です。 */
```

```
private static final int ADDRESS_MAX_LENGTH = 200;

/**
 * Servlet 初期化時に Oracle JDBC Driver を読み込みます。
 *
 * @throws ServletException JDBC Driver が見つからない場合
 */
@Override
public void init() throws ServletException {

    try {
        // Oracle JDBC Driver を読み込みます。
        Class.forName("oracle.jdbc.driver.OracleDriver");

    } catch (ClassNotFoundException e) {
        throw new ServletException("Oracle JDBC Driver が見つかりません。", e);
    }
}

/**
 * 登録ボタン押下時の処理です。
 *
 * @param req リクエスト
 * @param resp レスポンス
 * @throws ServletException Servlet エラー
 * @throws IOException 入出力エラー
 */
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {

    // POST で送られてきた日本語が文字化けしないようにします。
    req.setCharacterEncoding("UTF-8");

    // confirm.jsp の hidden から送られてきた値を DTO にまとめます。
    OrderDto order = new OrderDto(
        req.getParameter("customerName"),
        req.getParameter("customerAddress"),
        req.getParameter("deliveryName"),
        req.getParameter("deliveryAddress")
    );
};
```

```

try {
    // 登録前にもう一度バリデーションします。
    // hiddenの値は改ざんできるため、登録直前にもチェックします。
    validateOrder(order);

    // DB登録を行い、登録された配送IDを受け取ります。
    String deliveryId = insertOrder(order);

    // 完了画面で配送IDを表示するためrequestに保存します。
    req.setAttribute("deliveryId", deliveryId);

    // 完了画面へ進みます。
    req.getRequestDispatcher("/WEB-INF/jsp/customer/complete.jsp")
        .forward(req, resp);

} catch (IllegalArgumentException e) {

    // バリデーションエラーの場合は入力画面へ戻します。
    req.setAttribute("errorMessage", e.getMessage());
    req.setAttribute("order", order);

    req.getRequestDispatcher("/WEB-INF/jsp/customer/order.jsp")
        .forward(req, resp);

} catch (SQLException e) {

    // DBエラーの場合はServletExceptionとして投げます。
    throw new ServletException("配送依頼の登録に失敗しました。", e);
}
}

/**
 * 配送依頼情報をDBに登録します。
 *
 * <p>
 * ここがServiceの「登録処理の流れ」と、
 * Repositoryの「SQL実行」の両方を担当しています。
 * </p>
 *
 * @param order 配送依頼情報

```

```
* @return 登録された配送ID
* @throws SQLException DB処理に失敗した場合
*/
private String insertOrder(OrderDto order) throws SQLException {

    Connection conn = null;

    try {
        // DB接続を取得します。
        conn = DriverManager.getConnection(URL, USER, PASSWORD);

        // 自動コミットをOFFにします。
        // 2つのINSERTをまとめて成功・失敗させるためです。
        conn.setAutoCommit(false);

        // 配送IDを採番します。
        String deliveryId = getNextDeliveryId(conn);

        // delivery_orderに登録します。
        insertDeliveryOrder(conn, deliveryId, order);

        // delivery_situationに初期状態に登録します。
        insertDeliverySituation(conn, deliveryId);

        // ここまで成功したらDBに確定します。
        conn.commit();

        // 完了画面で表示するため、配送IDを返します。
        return deliveryId;

    } catch (SQLException e) {

        // 途中で失敗した場合は、登録を取り消します。
        if (conn != null) {
            conn.rollback();
        }

        throw e;

    } finally {
```

```

        // DB 接続を閉じます。
        if (conn != null) {
            conn.close();
        }
    }
}

/**
 * delivery_seq から次の配送IDを取得します。
 *
 * @param conn DB 接続
 * @return 配送ID
 * @throws SQLException SQL エラー
 */
private String getNextDeliveryId(Connection conn) throws SQLException {

    String sql = "SELECT LPAD(delivery_seq.NEXTVAL, 6, '0') AS delivery_id FROM dual";

    try (PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        if (rs.next()) {
            return rs.getString("delivery_id");
        }

        throw new SQLException("配送IDの採番に失敗しました。");
    }
}

/**
 * delivery_order テーブルに配送依頼情報を登録します。
 *
 * @param conn DB 接続
 * @param deliveryId 配送ID
 * @param order 配送依頼情報
 * @throws SQLException SQL エラー
 */
private void insertDeliveryOrder(Connection conn, String deliveryId, OrderDto order)
    throws SQLException {

    String sql =

```

```
        "INSERT INTO delivery_order ("
+ "delivery_id, "
+ "customer_name, "
+ "customer_address, "
+ "delivery_name, "
+ "delivery_address"
+ ") VALUES (?, ?, ?, ?, ?)";
```

```
try (PreparedStatement ps = conn.prepareStatement(sql)) {
```

```
    ps.setString(1, deliveryId);
    ps.setString(2, order.getCustomerName());
    ps.setString(3, order.getCustomerAddress());
    ps.setString(4, order.getDeliveryName());
    ps.setString(5, order.getDeliveryAddress());
```

```
    ps.executeUpdate();
```

```
}
```

```
}
```

```
/**
```

```
 * delivery_situation テーブルに配送状況の初期値を登録します。
```

```
 *
```

```
 * @param conn DB 接続
```

```
 * @param deliveryId 配送ID
```

```
 * @throws SQLException SQL エラー
```

```
 */
```

```
private void insertDeliverySituation(Connection conn, String deliveryId)
    throws SQLException {
```

```
    String sql =
```

```
        "INSERT INTO delivery_situation ("
+ "delivery_id, "
+ "delivery_status"
+ ") VALUES (?, ?)";
```

```
try (PreparedStatement ps = conn.prepareStatement(sql)) {
```

```
    ps.setString(1, deliveryId);
```

```
    // 初期状態です。
```

```

        // jokyo_mst の'10'が「営業所」の想定です。
        ps.setString(2, "10");

        ps.executeUpdate();
    }
}

/**
 * 配送依頼情報の入力チェックを行います。
 *
 * @param order 配送依頼情報
 */
private void validateOrder(OrderDto order) {

    if (order == null) {
        throw new IllegalArgumentException("配送依頼情報が取得できませんでした。");
    }

    validateRequired(order.getCustomerName(), "依頼主名");
    validateMaxLength(order.getCustomerName(), NAME_MAX_LENGTH, "依頼主名");

    validateRequired(order.getCustomerAddress(), "依頼主住所");
    validateMaxLength(order.getCustomerAddress(), ADDRESS_MAX_LENGTH, "依頼主住所");

    validateRequired(order.getDeliveryName(), "配送先名");
    validateMaxLength(order.getDeliveryName(), NAME_MAX_LENGTH, "配送先名");

    validateRequired(order.getDeliveryAddress(), "配送先住所");
    validateMaxLength(order.getDeliveryAddress(), ADDRESS_MAX_LENGTH, "配送先住所");
}

/**
 * 必須チェックを行います。
 *
 * @param value 入力値
 * @param itemName 項目名
 */
private void validateRequired(String value, String itemName) {

    if (isBlank(value)) {
        throw new IllegalArgumentException(itemName + "を入力してください。");
    }
}

```

```

    }
}

/**
 * 最大文字数チェックを行います。
 *
 * @param value 入力値
 * @param maxLength 最大文字数
 * @param itemName 項目名
 */
private void validateMaxLength(String value, int maxLength, String itemName) {

    if (value == null) {
        return;
    }

    if (value.length() > maxLength) {
        throw new IllegalArgumentException(itemName + "は" + maxLength + "文字以内で入力して
ください。");
    }
}

/**
 * 未入力かどうかを判定します。
 *
 * @param value 入力値
 * @return 未入力なら true
 */
private boolean isBlank(String value) {

    if (value == null) {
        return true;
    }

    return value.trim().replace(" ", "").isEmpty();
}
}

```